

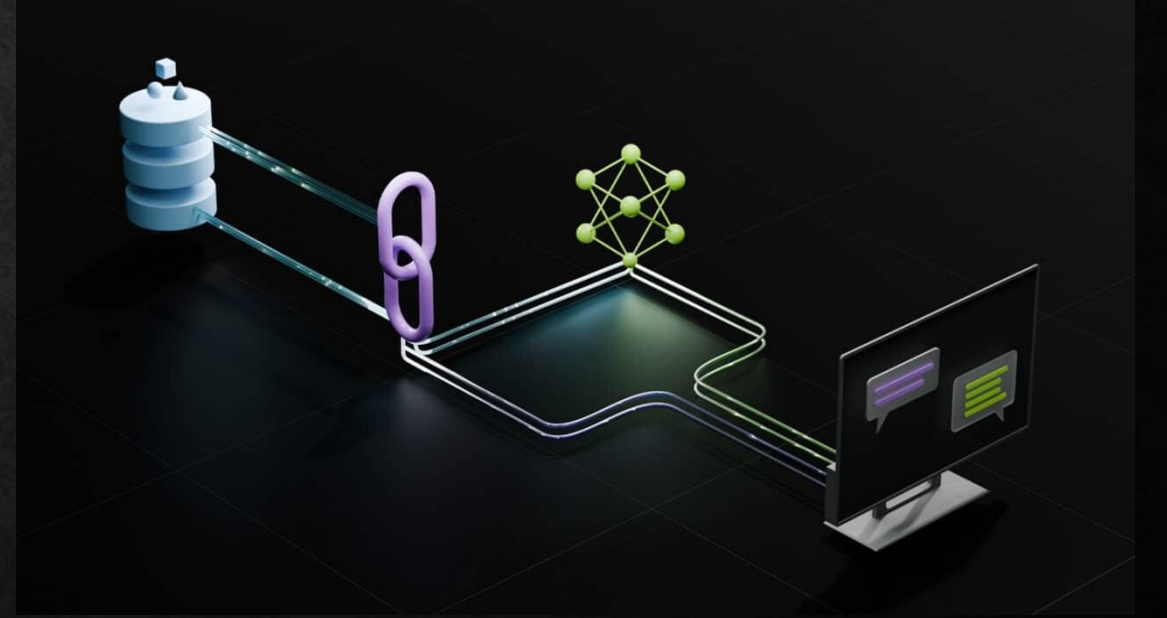
SUNUM 18: RAG (RETRİEVAL-AUGMENTED GENERATION)

AD-SOYAD : AYŞE HİLAL AKAR

ÖĞRENCİ NO : 2412705003

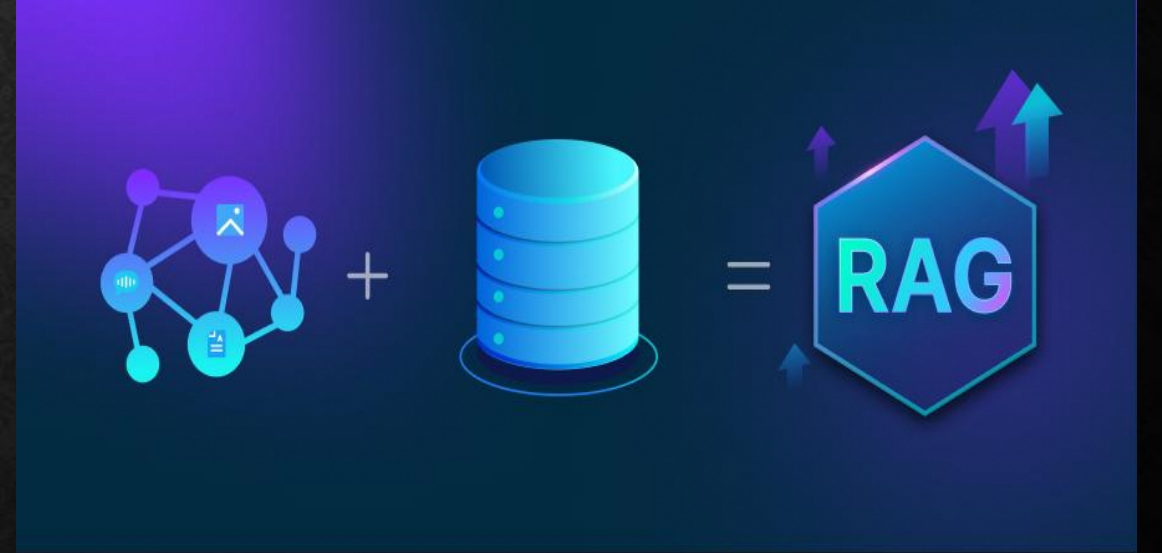
RAG (RETRIEVAL-AUGMENTED GENERATION) NEDİR?

- RAG Türkçe karşılığıyla 'Almayla Artırılmış Üretim' büyük dil modellerinin (LLM) dış veri kaynaklarından bilgi çekerek daha doğru, güncel ve bağlama uygun yanıtlar üretmesini sağlayan bir yapay zeka mimarisidir.
- RAG, doğal dil işleme alanında bilgi çekme(retrieval) ve dil üretimi(generation) yeteneklerini birleştiren bir yapıya sahiptir ve genellikle daha derin ve bilgi odaklı cevaplar üretmek için kullanılır.



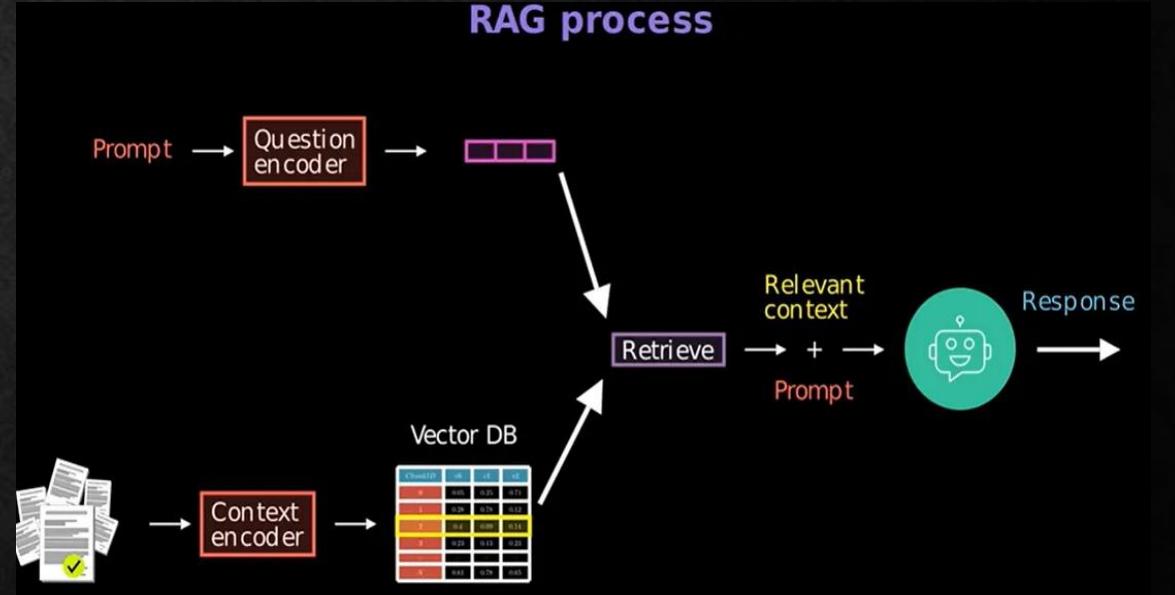
RAG NASIL ÇALIŞIR ?

- RAG'ın temel mantığı oldukça basittir: Bir yapay zeka modeli sadece eğitim sırasında öğrendiği bilgilerle sınırlı kalmak yerine, sorgu anında güvenilir veri kaynaklarından güncel bilgileri alır ve bu bilgileri yanıt üretirken kullanır. Bu yaklaşım, iki farklı modelin güçlü yönlerini birleştirir.



RAG NASIL ÇALIŞIR ?

- Birinci bileşen olan Retrieval (Bilgi Çekme) Modeli, belirli bir sorguya en uygun bilgi parçalarını büyük veri tabanlarından, belgelerden veya web kaynaklarından bulmakla görevlidir. İkinci bileşen Generative (Üretim) Modeli ise, bulunan bu bilgileri kullanarak akıcı, tutarlı ve bağlama uygun yanıtlar oluşturur.



RAG ÇALIŞMA BASAMAKLARI

- 1) Kaynakları Topla (Gather Sources)
- 2) Chunking: Dokümanı Parçalara Ayır
- 3) Embedding: Metni Vektöre Dönüştür
- 4) Vector DB: Vektörleri Depola
- 5) Kullanıcı Prompt'unu da Vektöre Çevir
- 6) Retrieval: Benzerlik Oranı En Yüksek Chunk Bulunur
- 7) Augmented Prompt: Soru + Bağlamı Birleştir
- 8) LLM Yanıtı Üretir (Generate)

Context encoding

Chunk 0

Chunk 1

Chunk 2

Chunk 3

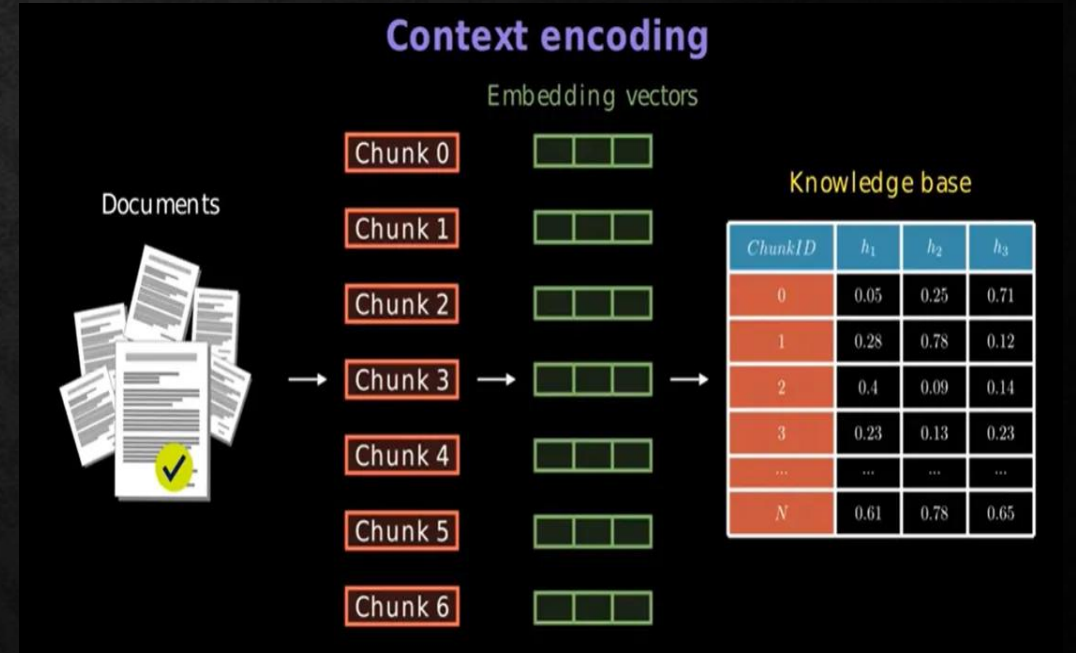
Chunk 4

Chunk 5

Chunk 6

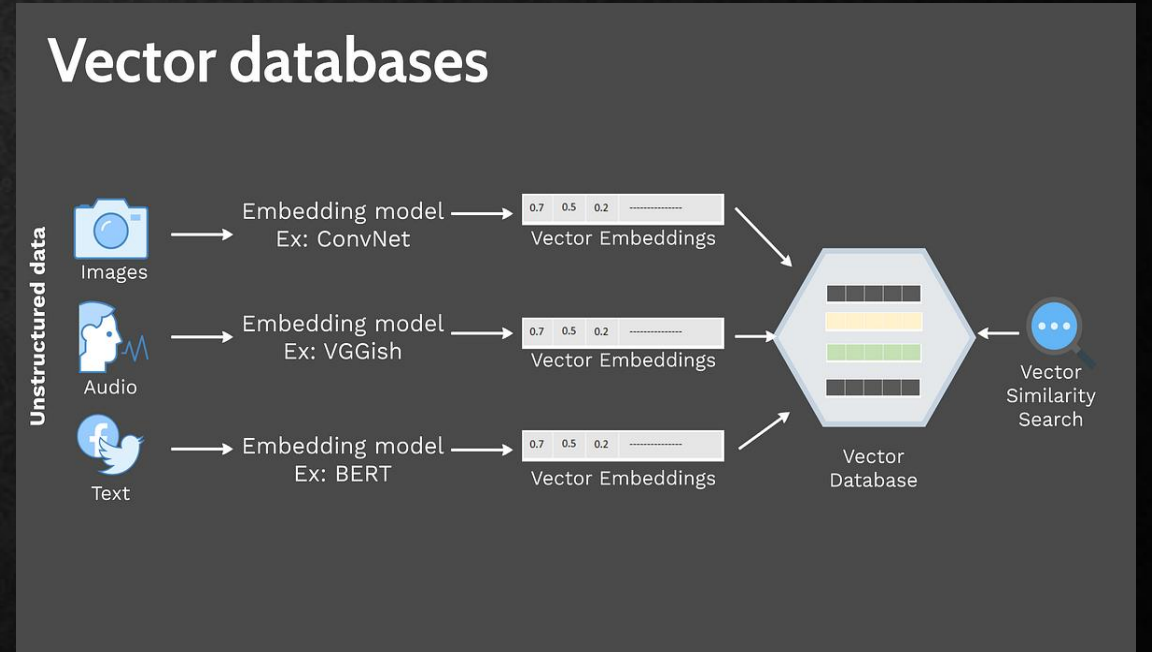
EMBEDDING

- Embedding, ham verileri daha anlaşılır ve işlenebilir bir forma dönüştüren güçlü bir araçtır.
- Veriler arasındaki anlamsal ilişkileri ortaya çıkararak modellerin daha etkili çalışmasını sağlar.
- Metnin anlamını sayılarla ifade eden bir vektördür.
- Örneğin bir chunk şu olabilir= Mobil politikamız, çalışanların kişisel cihazlarını iş için kullanmalarına izin verir.
- Metin embeddingten geçer = $[0.23, 0.11, 0.78, \dots]$



VECTOR DATABASE

- Vektör veritabanı, vektör gömmeleri adı verilen özel veri türlerini depolamak ve aramak için oluşturulmuş bir veritabanı türüdür. Bu gömmeler, metin, görüntü, video veya ses gibi şeylerin anlamını veya özelliklerini temsil eden sayılardır.
- Ana işleri birbirlerine benzeyen şeyleri -benzerlik arama olarak bilinen- tam eşleşme olmasa bile, gömmelerinin matematiksel uzayda ne kadar yakın olduğunu karşılaştırarak hızlıca bulmaktır.

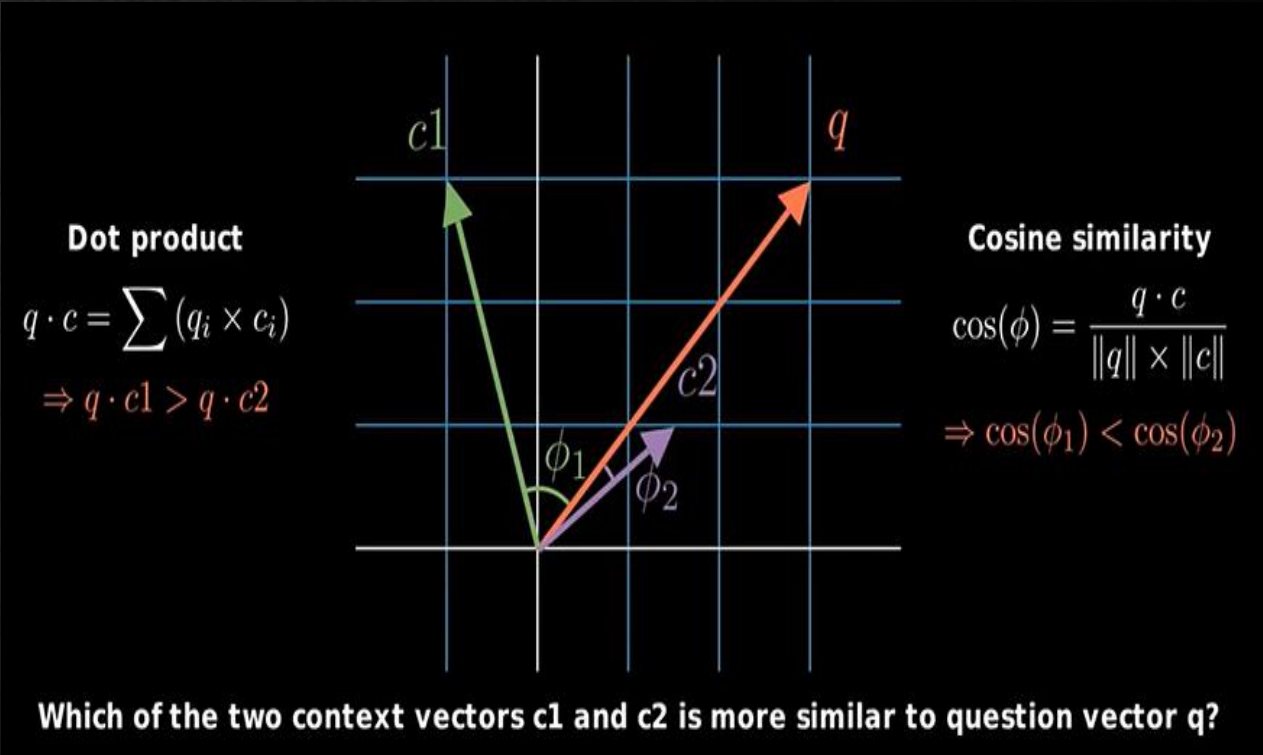


RETRIEVAL

- Sistem, soru vektörü ile doküman vektörleri arasındaki benzerliği ölçer.
- RAG sistemlerinin merkezinde şu soru vardır:
“Kullanıcının sorduğu soru ile, milyonlarca satırlık kurumsal doküman arasından hangi parçalar anlam olarak en çok örtüşüyor?”
- Bu sorunun cevabı, metinleri önce embedding vektörlerine dönüştürmek, sonra da bu vektörlerin birbirine ne kadar benzediğini ölçmekten geçer.
- Bu benzerliği ölçmek için iki temel araç kullanıyoruz:

❖ **Dot Product (Skaler Çarpım)**

❖ **Cosine Similarity (Kosinüs Benzerliği)**



RETRIEVAL

Dot product, vektörlerin bileşenlerini çarpıp toplayarak hesaplanır. Vektörlerin büyüklük ve yönüne bakar.

- $\text{dot_product}(a,b) = \sum(a_i \times b_i) = a_1b_1 + a_2b_2 + \dots + a_nb_n$
Geometrik olarak da şöyle ifade edilir:
 $\text{dot_product}(a,b) = |a| \times |b| \times \cos(\theta)$
- İki vektör hem aynı yönde hem de büyükse → büyük bir dot product değeri çıkar.
- Yönleri farklıysa → dot product düşer.
- Tam tersi yönlerdeyse → dot product negatif olabilir.
- Ancak dot product sadece yönü değil, vektörlerin büyüklüğünü de etkiler. Bu yüzden uzun vektörler bazen olduğundan daha benzer görünebilir.

Cosine similarity, İki vektörün sadece yönüne bakalım, uzunluğu önemli değil. Vektörler arasındaki açığa bakar.

- $\text{cosine_similarity}(a,b) = \text{dot_product}(a,b) / (|a| \times |b|)$
- Sonuç -1 ile +1 arasında çıkar:
- +1 → tamamen aynı yön (çok benzer anlam)
- 0 → dik açı (ilişki yok)
- -1 → zıt yön (tam tersi anlam)
- Bu yöntem özellikle metin analizinde kullanılır çünkü metnin uzunluğundan çok anlam benzerliğini ölçer. Bu yüzden RAG ve embedding sistemlerinde en yaygın benzerlik ölçülerinden biridir.

-COSİNE SİMİLARİTY ÖRNEK

- Diyelim ki embedding modeline üç kelime veriyoruz:
- “doktor” → bir vektör
- “hemşire” → bir vektör
- “elma” → bir vektör
- “doktor” ve “hemşire” kelimeleri çoğu cümlede aynı bağlamlarda geçer

❑ doctor vs nurse → 0.91 (çok yüksek)

❑ doctor vs apple → 0.08 (çok düşük)

- RAGDAKİ KULLANIMI VE DEVAMI

- Retriever şu işlemi yapar:
 1. Soru vektörü ile her chunk vektörünün cosine similarity'sini hesaplar.
 2. En yüksek benzerliğe sahip ilk K chunk'ı seçer
 3. LLM sadece bu bağlam üzerinden yanıt üretir
- Retriever tarafından dönen metin parçaları ile kullanıcının orijinal sorusu birleştirilir.
- Bu yeni metne **augmented prompt** denir.
- Ve artık LLM istediği bilgileri görebilir.

GERÇEK HAYATLA İLİŞKİLENDİRİLEN ÖRNEKLER

ÖRNEK 1

Mesela bir üniversitenin chatbotunu düşünelim.

Öğrenci:

“Vize sınavı ne zaman?”

diye soruyor.

Süreç şöyle işler:

- 1.Kullanıcının sorusu embedding'e çevrilir.
- 2.Benzer belgeler cosine similarity ile bulunur.
- 3.En alakalı bilgiler yapay zekâya verilir.
- 4.Yapay zekâ bu bilgilere dayanarak cevap oluşturur.

Kısaca RAG:

- Bilgiyi ezberden üretmek yerine,
- Önce doğru kaynağı bulup,
- Sonra cevap üretme yöntemidir.

Retrieval Augmented Generation (RAG)



User Asks
a Question



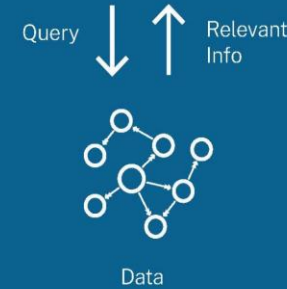
LLM Smart
Search



LLM Question +
Relevant Info

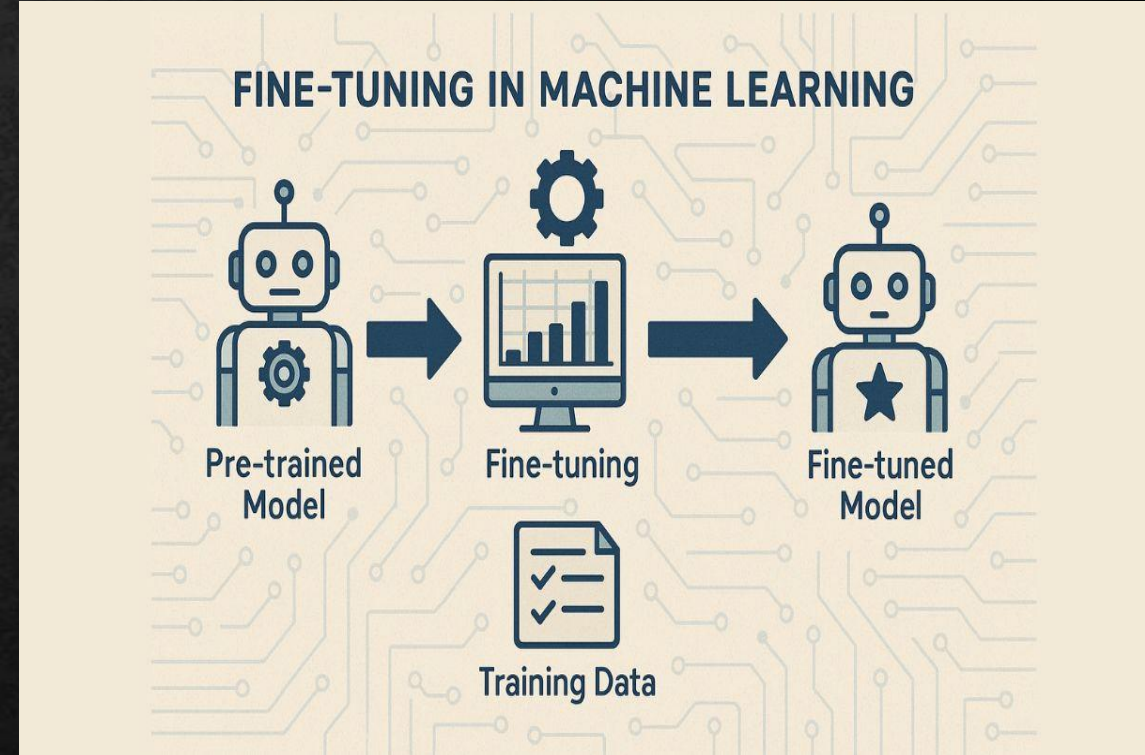


Enriched
Answer



FINE-TUNING

- Fine-tuning (ince ayar) belirli bir görevdeki performansı artırmak için önceden eğitilmiş bir modelin parametrelerini ayarlama işlemidir.
- LLM'lerin belirli görevler için yeteneğini geliştirmek üzere etiketli örneklerden oluşan bir veri kümesi ile güncellendiği bir denetimli öğrenme sürecidir.
- LLM'lerin benzersiz iş ihtiyaçlarına uyacak ve en iyi performansı gösterecek şekilde uyarlanmasına olanak tanıdığından, transfer öğrenimi yoluyla LLM'lerin geliştirilmesinde önemli bir adımdır.



RAG VS FİNE-TUNİNG

- **RAG:** Dış bilgi kaynakları kullanır, veri güncellemesi kolaydır, daha düşük maliyetlidir. Güncel ve spesifik bilgi gerektiren uygulamalar için idealdir.
- **Fine-tuning:** Modelin ağırlıklarını günceller, belirli bir görev veya stil için optimize eder, daha yüksek maliyetlidir. Tutarlı ton ve format gerektiren uygulamalar için uygundur.
- 2026'da en iyi sonuçlar genellikle RAG ve fine-tuning'in birlikte kullanılmasıyla elde edilmektedir.



KAYNAKÇA

- <https://ekolsoft.com/tr/b/rag-sistemi-nedir-retrieval-augmented-generation-rehberi>
- <https://blog.openzeka.com/ai/rag-retrieval-augmented-generation-nedir/>
- <https://medium.com/@zehrasultanguruz/rag-retrieval-augmented-generation-embedding-vector-db-cosine-similarity-ragin-anatomisi-9e631bf507c3>
- <https://www.komtas.com/post/rag-nedir-rag-nasil-calisir>
- <https://www.komtas.com/glossary/embedding-nedir>
- <https://www.sap.com/turkey/resources/what-is-a-vector-database>
- <https://www.cozumpark.com/fine-tuning-nedir/>

DİNLEDİĞİNİZ İÇİN TEŞEKKÜRLER